

```
:param url: URL for the new :class:`Request` object.
:param **kwargs: Optional arguments that ``request`` takes.
:rtype: requests.Response
"""
```

--snip--

The `sessions.py` file's `request` contains docstrings for the module ❷, the `Session` class ❸, and the `Session` class's `get()` method ❹. Note that although the module's docstring must be the first string to appear in the module, it should come after any magic comments, such as the shebang line or encoding definition ❶.

Later, you can retrieve the docstring for a module, class, function, or method by checking the respective object's `__doc__` attribute. For example, here we examine the docstrings to find out more about the `sessions` module, the `Session` class, and the `get()` method:

```
>>> from requests import sessions
>>> sessions.__doc__
'\nrequests.session\n~~~~~\n\nThis module provides a Session object to
manage and persist settings across\nrequests (cookies, auth, proxies).\n'
>>> sessions.Session.__doc__
"A Requests session.\n\n    Provides cookie persistence, connection-pooling, and
configuration.\n\n    Basic Usage::\n\n        >>> import requests\n--snip--
>>> sessions.Session.get.__doc__
'Sends a GET request. Returns :class:`Response` object.\n\n    :param url:
URL for the new :class:`Request` object.\n\n    :param **kwargs:
```

--snip--

Automated documentation tools can take advantage of docstrings to provide context-appropriate information. One of these tools is Python's built-in `help()` function, which displays the docstring of the object you pass it in a more readable format than the raw `__doc__` string directly. This is useful when you're experimenting in the interactive shell, because you can immediately pull up information on any modules, classes, or functions you're trying to use:

```
>>> from requests import sessions
>>> help(sessions)
```